

# Rapport de benchmark

BenchmarkTick-16 — analyse CPU, mémoire et hyperfine

|                 |                                                                                              |
|-----------------|----------------------------------------------------------------------------------------------|
| Généré le       | 2026-09-24T10:36:22.0309855Z                                                                 |
| Commande        | go test ./game -run '^\$' -bench '^BenchmarkTick\$' -benchmem -count 1 (BENCHMARK_WIDTH=1200 |
| Plateforme      | windows/amd64                                                                                |
| Processeur      | AMD Ryzen 7 7735HS with Radeon Graphics                                                      |
| Carte benchmark | 1200 × 1200 (densité 0.25)                                                                   |
| Profil CPU      | cpu-20260924-103622031.prof                                                                  |
| Profil mémoire  | memory-20260924-103622031.prof                                                               |

## 1. Résultats du benchmark

| Benchmark        | Moyenne   | ns/op     | B/op | allocs/op |
|------------------|-----------|-----------|------|-----------|
| BenchmarkTick-16 | 122.81 ms | 122807633 | 24   | 0         |

### Détail hyperfine

| Métrique                       | Valeur    |
|--------------------------------|-----------|
| Moyenne                        | 2.5984 s  |
| Écart-type                     | n/a       |
| Médiane                        | 2.5984 s  |
| Temps utilisateur              | 14.2294 s |
| Temps système                  | 1.2244 s  |
| Min                            | 2.5984 s  |
| Max                            | 2.5984 s  |
| Runs mesurés                   | 1         |
| Coefficient de variation       | n/a       |
| Pic WorkingSet commande Go     | 27.51 MiB |
| Pic mémoire privée commande Go | 61.09 MiB |

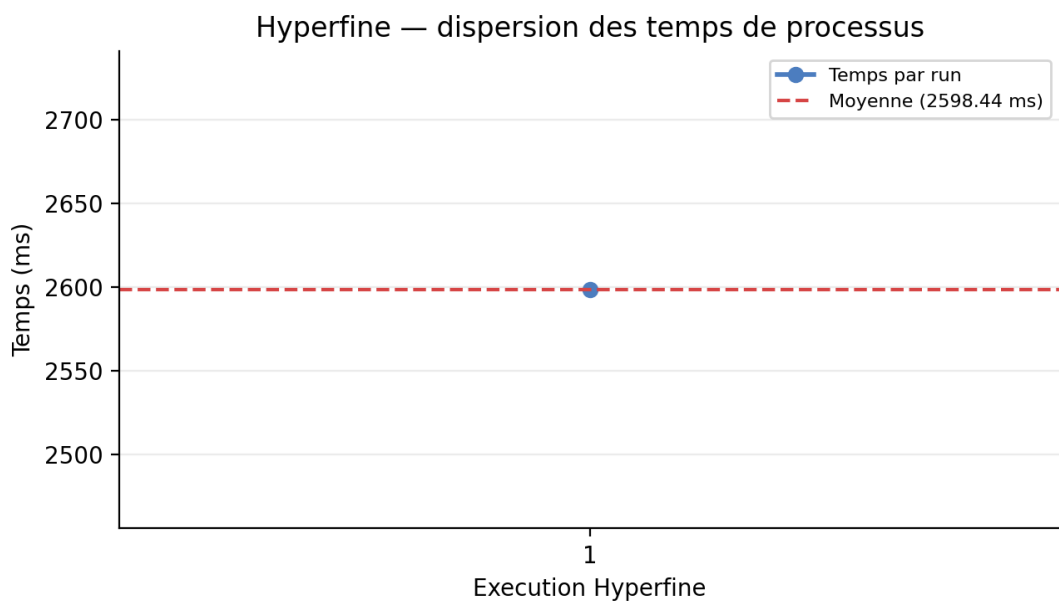


Figure — Temps de chaque exécution Hyperfine et moyenne.

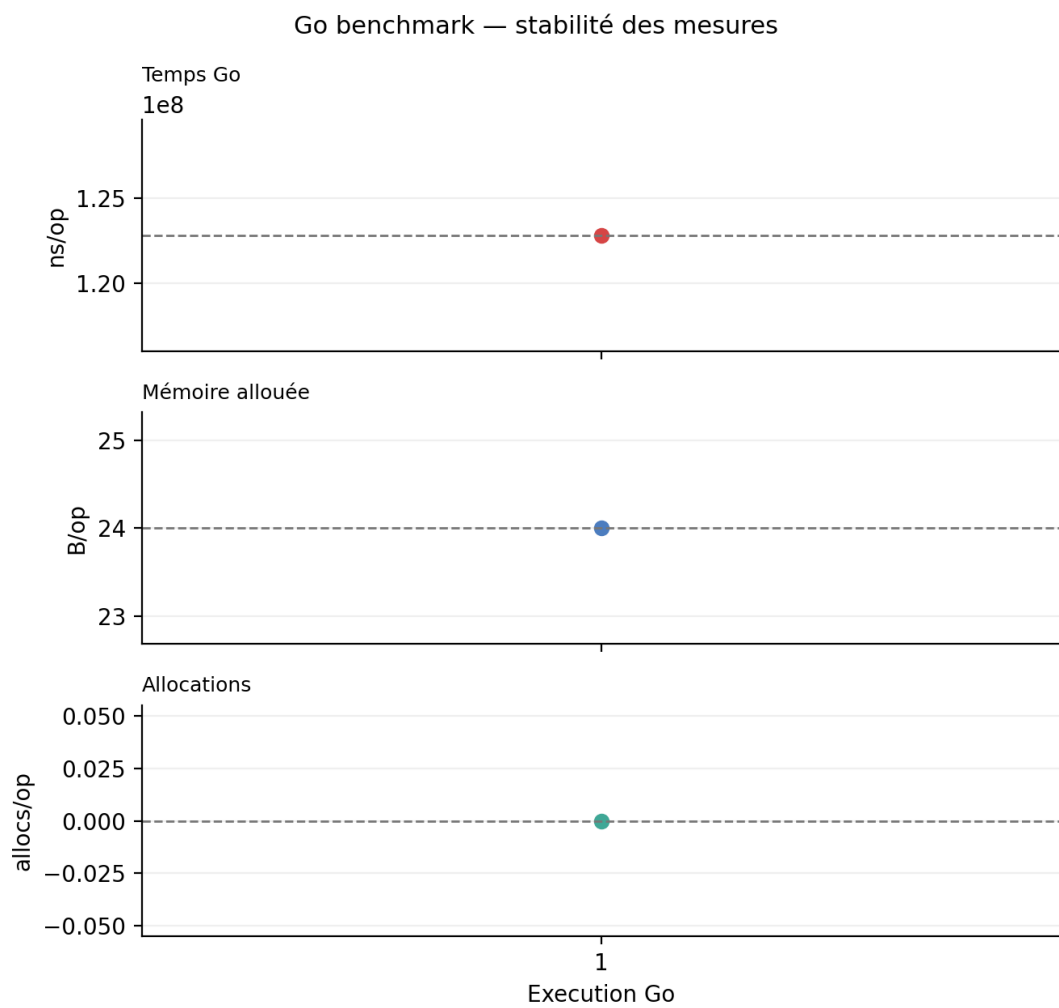


Figure — Dispersion des mesures Go : temps, mémoire allouée et allocations.

## Résumé CPU et mémoire

| Mesure                    | Valeur    |
|---------------------------|-----------|
| CPU échantillonné (pprof) | 13.97s    |
| Mémoire allouée (pprof)   | 104.92MB  |
| Heap vivant (pprof)       | 7176.97kB |
| RAM physique totale       | 32061 MiB |
| RAM physique disponible   | 14976 MiB |
| RAM physique utilisée     | 17085 MiB |
| Mémoire par opération     | 24 B/op   |

## Garbage collector Go

|                          |            |
|--------------------------|------------|
| Cycles GC                | 58         |
| Temps GC cumulé          | 62.430 ms  |
| Plus longue pause        | 2.720 ms   |
| Temps CPU GC cumulé      | 330.200 ms |
| Objectif mémoire maximal | 97.00 MB   |

La RAM physique est l'état du système au lancement du benchmark. La mémoire pprof mesure le heap Go échantillonné (alloué ou encore vivant), pas toute la mémoire RSS du processus. Hyperfine peut fournir le RSS quand le backend système le supporte ; une valeur nulle est donc signalée comme indisponible.

## 2. Profil CPU — où passe le temps ?

Type de profil : **cpu**. Total échantillonné : **13.97s**.

| Fonction                                       | Flat   | Flat % | Cumulé | Cumulé % |
|------------------------------------------------|--------|--------|--------|----------|
| jeu-de-la-vie/game.(*Board).infectionCandidate | 13.37s | 95.71% | 13.77s | 98.57%   |
| runtime.asyncPreempt                           | 0.40s  | 2.86%  | 0.40s  | 2.86%    |
| jeu-de-la-vie/game.startCandidatePool.func1    | 0.08s  | 0.57%  | 13.85s | 99.14%   |
| jeu-de-la-vie/game.(*Board).Step               | 0.06s  | 0.43%  | 0.07s  | 0.50%    |
| jeu-de-la-vie/game.BenchmarkTick               | 0      | 0.00%  | 0.09s  | 0.64%    |
| testing.(*B).launch                            | 0      | 0.00%  | 0.07s  | 0.50%    |
| testing.(*B).runN                              | 0      | 0.00%  | 0.09s  | 0.64%    |

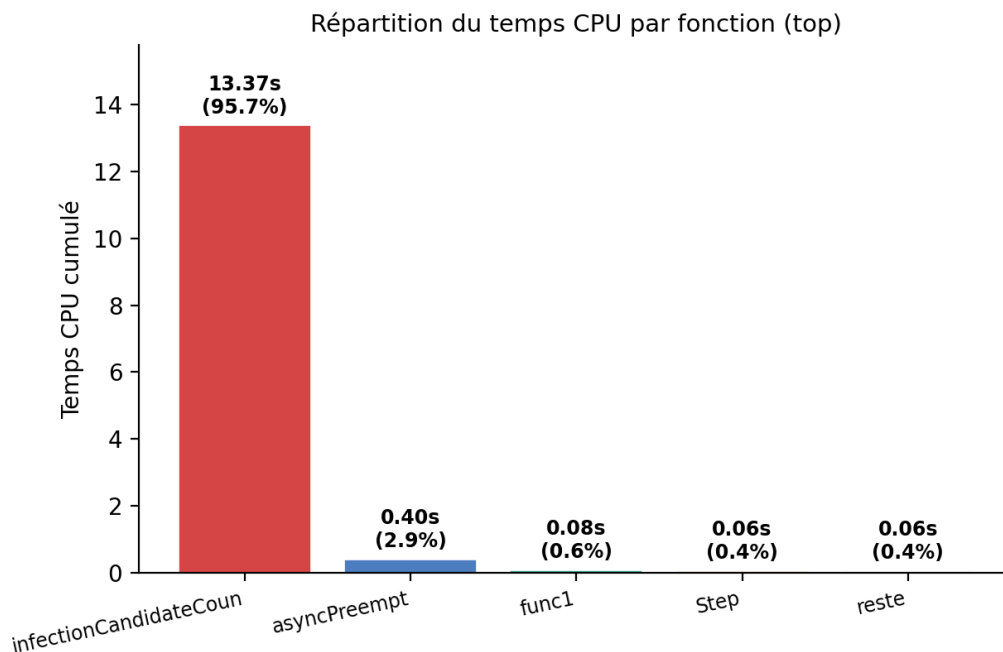


Figure — Répartition du temps CPU par fonction (temps flat, top fonctions).

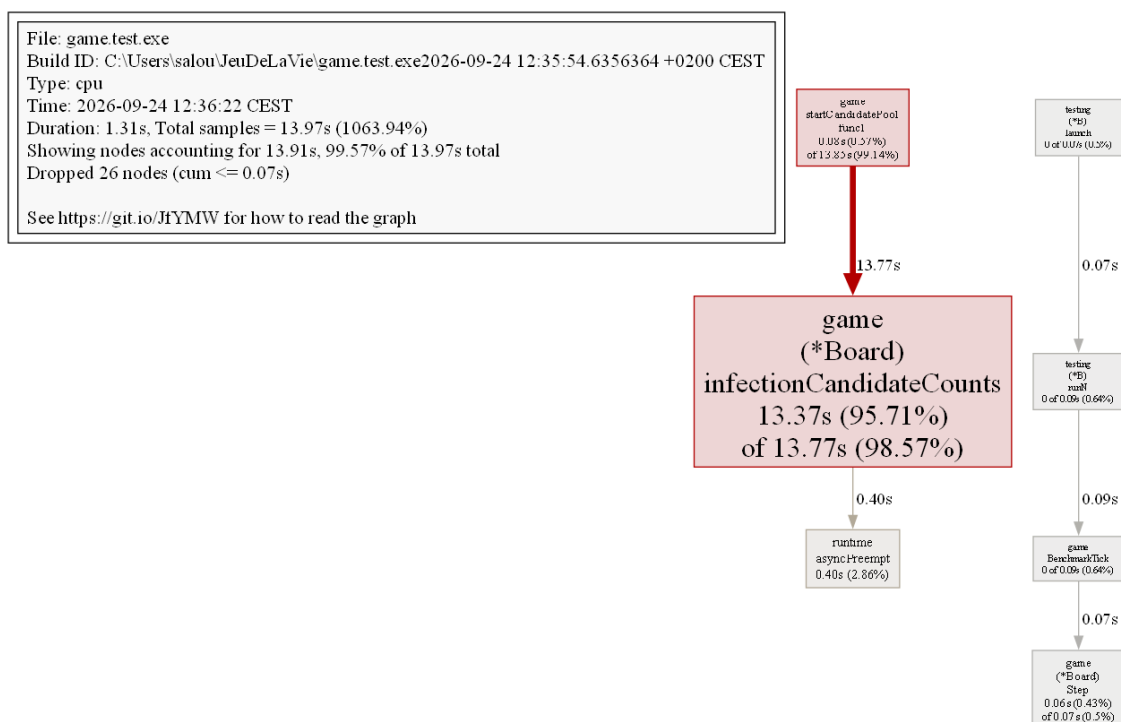


Figure — Graphe d'appels CPU (go tool pprof -png).

## 2.1 Diagnostic automatique

La fonction dominante de ce run est **jeu-de-la-vie/game.(\*Board).infectionCandidateCounts**, responsable de **95.7%** du temps CPU total (flat), pour **98.6%** cumulé avec ses appels. C'est la cible prioritaire d'optimisation pour ce run.

- Le nom de cette fonction suggère une recherche/un parcours répété (distance, voisinage, scan...) — vérifier si elle itère sur l'ensemble des données à chaque appel plutôt que sur un sous-ensemble borné (structure spatiale : grille, quadtree, index).
- Comparer ce hotspot avec le run précédent : s'il a changé de fonction, l'optimisation précédente a fonctionné et un nouveau goulot est apparu ailleurs.

- Vérifier s'il existe un algorithme en  $O(n)$  ou  $O(\log n)$  équivalent pour cette opération avant d'optimiser au niveau micro (inlining, allocations).

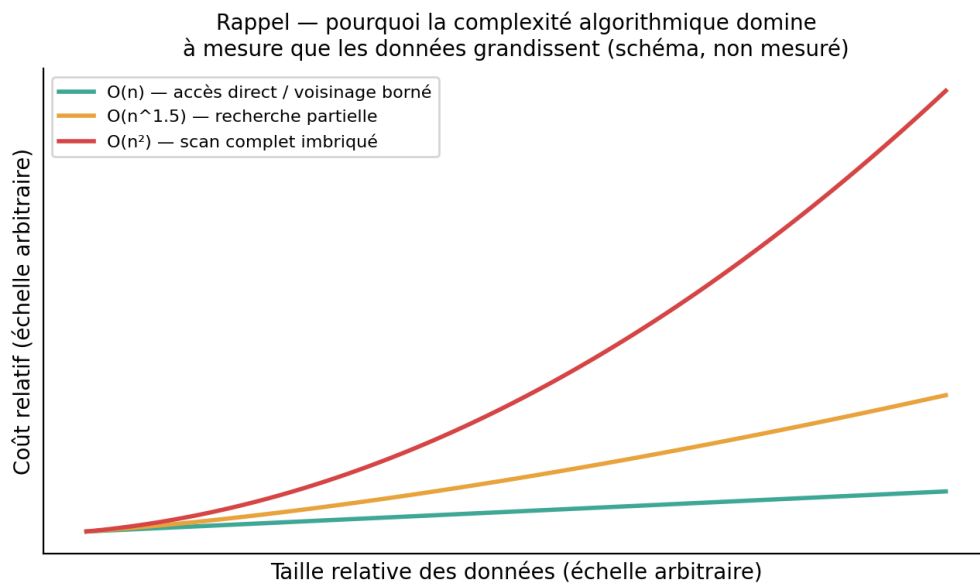


Figure — Rappel pédagogique sur la complexité algorithmique (schéma générique).

### 3. Profil mémoire — allocations

Type de profil : `alloc_space`. Total : **104.92MB**.

| Fonction                                    | Flat    | Flat % | Cumulé  | Cumulé % |
|---------------------------------------------|---------|--------|---------|----------|
| jeu-de-la-vie/game.NewBoard                 | 71.83MB | 68.46% | 71.83MB | 68.46%   |
| jeu-de-la-vie/game.RandomBoard              | 21.98MB | 20.95% | 93.81MB | 89.41%   |
| runtime.mallocgc                            | 4.51MB  | 4.30%  | 4.51MB  | 4.30%    |
| jeu-de-la-vie/game.startCandidatePool.func1 | 2MB     | 1.91%  | 2MB     | 1.91%    |
| runtime/pprof.StartCPUProfile               | 1.16MB  | 1.10%  | 1.16MB  | 1.10%    |
| compress/flate.NewWriter (inline)           | 0.88MB  | 0.84%  | 1.95MB  | 1.85%    |
| compress/flate.(*compressor).init           | 0.53MB  | 0.51%  | 1.06MB  | 1.01%    |
| compress/flate.newDeflateFast (inline)      | 0.53MB  | 0.51%  | 0.53MB  | 0.51%    |

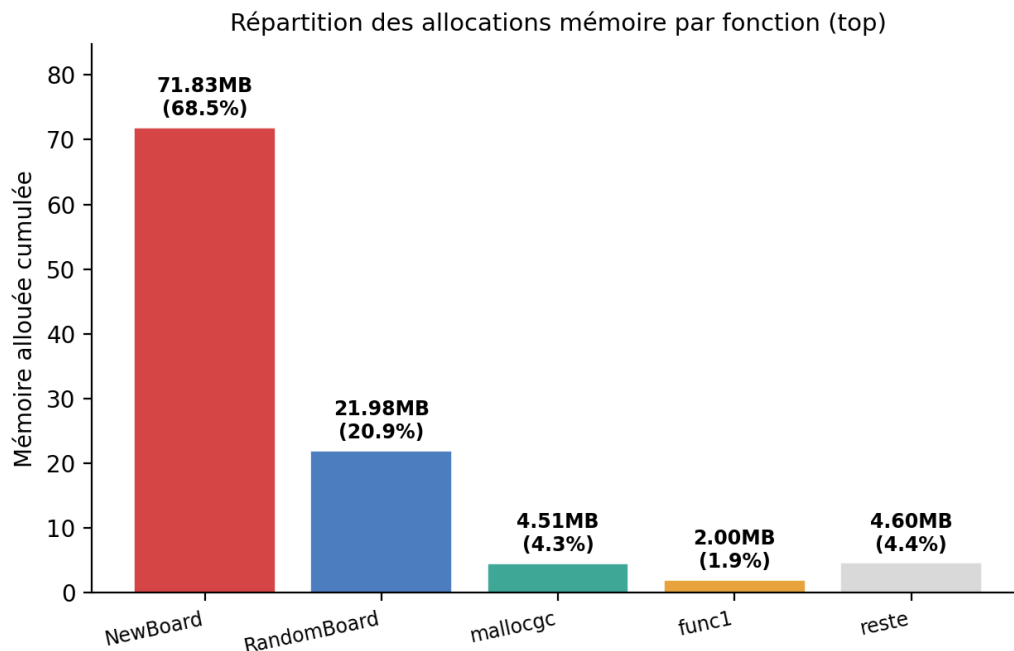


Figure — Répartition des allocations mémoire par fonction.

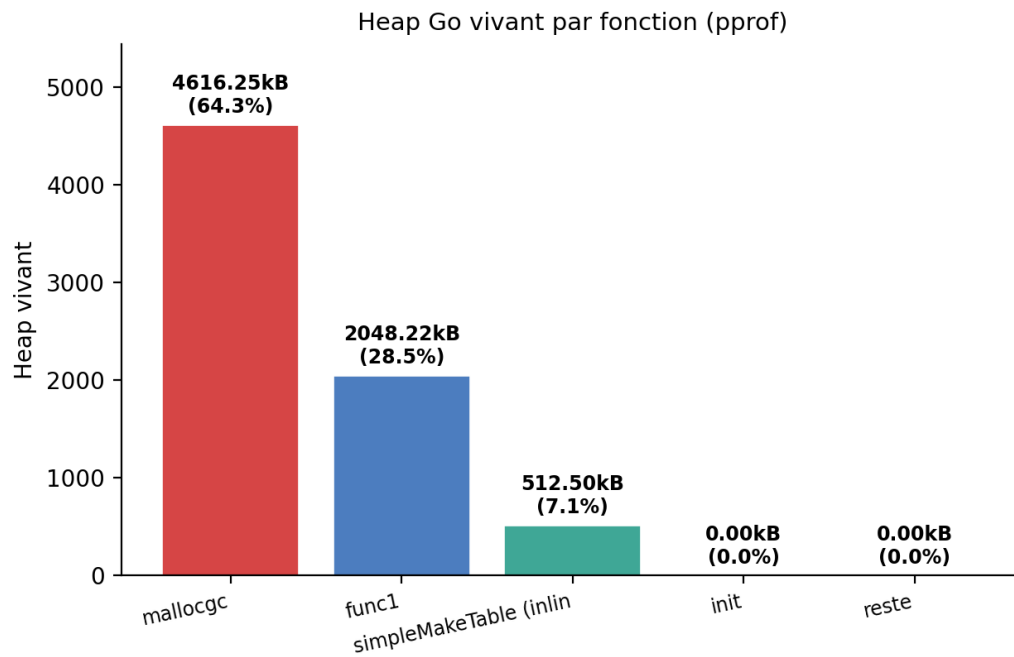


Figure — Heap Go encore vivant par fonction (pprof -inuse\_space).

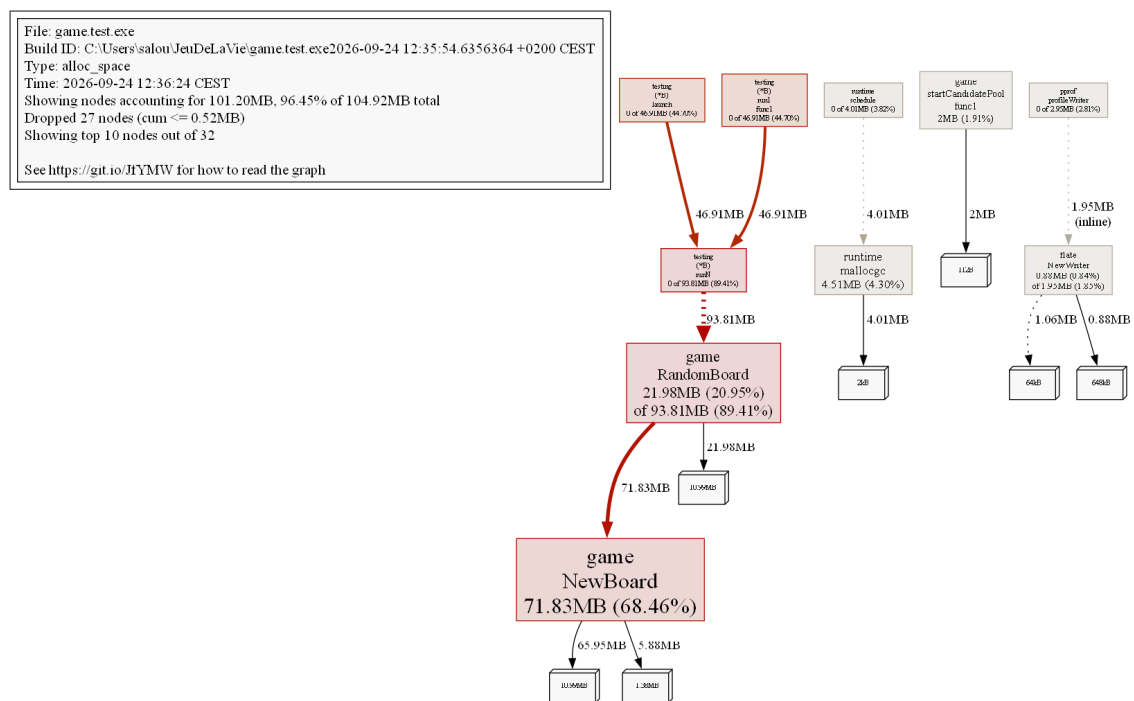


Figure — Graphe d'appels mémoire (go tool pprof -png).

## 4. Limites de ce benchmark & informations manquantes

Générées automatiquement à partir de la configuration détectée :

- **Peu de runs (1).** Avec moins de 5 exécutions, aucune médiane/écart-type fiable n'est disponible. Recommandé :  $\geq 10$  runs (-count=10 ou hyperfine --min-runs 10).
- **Aucun warmup.** La première exécution peut être biaisée (cache froid, JIT du GC non stabilisé). Ajouter 1-2 warmups avant la mesure.
- **Pas de comparaison avant/après** intégrée à ce rapport — conservez les PDF successifs pour comparer les hotspots et le temps moyen dans le temps.
- **Un seul point de taille de données.** Si applicable, mesurer plusieurs tailles permettrait de vérifier empiriquement la complexité algorithmique observée en section 2.

### Notes de l'outil de benchmark

- Mesure du calcul du tick sur une carte  $1200 \times 1200$ .
- Densité de la carte : 0.25 ; seed : 42.
- La latence API est mesurée à part dans le frontend, sans rendu canvas.
- La trace GC détaillée est enregistrée dans le fichier `gc-\*.log` et résumée dans le PDF.
- Les métriques I/O, réseau et base de données ne sont pas instrumentées dans ce benchmark.

---

Rapport généré automatiquement le 2026-09-24 12:36:41 par generate\_benchmark\_report.py, à partir des fichiers les plus récents du dossier de benchmarks. Analyse réalisée avec go tool pprof.